

In entire web application ,the DAO layer usually consists of a lot of boilerplate code that can be simplified.

Benefits of simplification

1. Decrease in the number of layers that we need to define and maintain
2. Consistency of data access patterns and consistency of configuration.

Spring Data JPA framework takes this simplification one step forward and makes it possible to remove the DAO implementations entirely. The interface of the DAO is now the only artifact that we need to explicitly define.

For this , a DAO interface needs to extend the JPA specific Repository interface – JpaRepository or its super i/f CrudRepository. This will enable Spring Data to find this interface and automatically create an implementation for it.

Spring Data JPA Links (Bookmark them!)

1. API Docs : <https://docs.spring.io/spring-data/jpa/docs/2.7.18/api/>
2. Ref docs : <https://docs.spring.io/spring-data/jpa/docs/2.7.18/reference/html/>

1. Inherited API

API

Root i/f

`org.springframework.data.repository.Repository<T, ID>` : a marker i/f

T : Type of the entity

ID : type of ID property

2. Sub i/f

`org.springframework.data.repository.CrudRepository<T, ID>`

By extending from this interface we get the most required CRUD methods in a standard DAO.

eg : CrudRepository methods

`public long count()`

Returns the number of entities available.

`void delete(T entity)`

Deletes a given entity.

`void deleteAll()`

Deletes all entities managed by the repository.

`void deleteAll(Iterable<? extends T> entities)`

Deletes the given entities.

`void deleteById(ID id)`

Deletes the entity with the given id.

`boolean existsById(ID id)`

Returns whether an entity with the given id exists.

`Iterable<T> findAll()`

Returns all instances of the type.

`Iterable<T> findAllById(Iterable<ID> ids)`

Returns all instances of the type with the given IDs.

`Optional<T> findById(ID id)`

Retrieves an entity by its id. (For actual retrieval , use Optional class's API orElseThrow)

`<S extends T>`

`S save(S entity)`

Saves or updates a given entity(same as Hibernate Session's : merge)

I/P : transient/detached entity

O/P : a persistent entity

If you pass

1. null ID : insert query (save)
2. non null ID BUT matching rec in DB : select ,update
3. non null ID BUT no matching rec in DB : select , insert

<S extends T>

Iterable<S> saveAll(Iterable<S> entities)

Saves all given entities.

3. Sub i/f

o.s.data.repository.PagingAndSortingRepository<T, ID>

Methods

Iterable<T> findAll(Sort sort);

Page<T> findAll(Pageable pageable);

Used for sorting n pagination(=dividing the larger no of results in to fixed size pages)

4. Sub i/f

o.s.data.jpa.repository.JpaRepository<T, ID>

This will be the super i/f of all DAO layer interfaces.

eg : public interface ProductDao extends JpaRepository<Product,Long>

{....}

Methods of JpaRepository

void deleteAllInBatch()

Deletes all entities in a batch call.

void deleteInBatch(Iterable<T> entities)

Deletes the given entities in a batch which means it will create a single Query.

List<T> findAll()

Returns List of persistent entities.

<S extends T>

List<S> findAll(Example<S> example)

<S extends T>

List<S> findAll(Example<S> example, Sort sort)

List<T> findAll(Sort sort)

List<T> findAllById(Iterable<ID> ids)

void flush()

Flushes all pending changes to the database.

T getOne(ID id)

Returns a reference to the entity with the given identifier.

<S extends T>

List<S> saveAll(Iterable<S> entities)

Spring Data JPA framework supplies readymade implementation classes for above mentioned interfaces (eg - CrudRepository, PagingAndSortingRepository, JpaRepository)

Spring Data offers support at 3 different levels.

1. Ready made implementation of the Inherited API

(eg - SimpleJpaRepository - implementation class)

2. Derived Query methods(Finder methods typically !) : based upon Method proxy pattern

eg : For User's signin , add in UserDao(or UserRepository) i/f (extending JpaRepository)
Optional<User> findByEmailAndPassword(String em,String pass);
//How to get list of users born in the range - begin n end date

By extending one of the Repository interfaces, the DAO will already have some basic CRUD methods (and queries) defined and implemented.

To define more specific access methods, Spring JPA supports quite a few options:

1. simply define a new method in the interface

provide the actual JPQL query by using the @Query annotation

When Spring Data creates a new Repository implementation, it analyses all the methods defined by the interfaces and tries to automatically generate queries from the method names. While this has some limitations, it's a very powerful and elegant way of defining new custom access methods with very little effort.

eg :

```
Customer findByName(String name);  
List<Person> findByAddressAndLastname(String address, String lastname);
```

// Enables the distinct flag for the query

```
List<Person> findDistinctPeopleByLastnameOrFirstname(String lastname, String firstname);
```

// Enabling ignoring case for an individual property

```
List<Person> findByLastnameIgnoreCase(String lastname);
```

// Enabling static ORDER BY for a query

```
List<Person> findByLastnameOrderByFirstnameAsc(String lastname);
```

```
List<Person> findByAddressZipCode(String zipCode);
```

Assuming a Person p has an Address with a String zipCode. In that case, the method creates the property traversal p.address.zipCode.

Limiting the result size of a query with Top and First

```
User findFirstByOrderByLastnameAsc();
```

```
User findTopByOrderByAgeDesc();
```

```
Page<User> queryFirst10ByLastname(String lastname, Pageable pageable);
```

```
Slice<User> findTop3ByLastname(String lastname, Pageable pageable);
```

```
List<User> findFirst10ByLastname(String lastname, Sort sort);
```

```
List<User> findTop10ByLastname(String lastname, Pageable pageable);
```

3

Custom query methods

One can add directly in DAO i/f, using JPQL

eg :

```
@Query("select u from User u where u.emailAddress = :em")
Optional<User> fetchUserByEmailAddress(@Param("em")String emailAddress);
```

OR (short cut)

```
@Query("select u from User u where u.emailAddress = :em")
Optional<User> fetchUserByEmailAddress(String em);
```

```
@Query("SELECT p FROM Person p WHERE LOWER(p.name) = LOWER(:nm)")
Optional<Person> retrieveByName(String nm);
```

@Modifying

```
@Query("delete from User u where u.role.id = :id")
int deleteInBulkByRoleId(long id);
```

@Modifying

```
@Query("update User u set u.firstname = ?1 where u.lastname = ?2")
int setFixedFirstnameFor(String firstname, String lastname);
```

3. Transaction Configuration

The actual implementation of the Spring Data managed DAO is hidden since we don't work with it directly. It's implemented by – the SimpleJpaRepository – which defines default transaction mechanism using annotations.

These can be easily overridden manually per method.

4. Exception Translation is still supplied

Exception translation is still enabled by the use of the @Repository annotation internally applied on the DAO implementation class.